

InjectionPlugin 使用说明

项目结构

```
task2/
├─ injection-gradle-plugin/    # Gradle 插件源码
│   └─ src/main/kotlin/injection/
│       ├── InjectionPlugin.kt    # 插件入口
│       ├── bytecode/            # 字节码注入 (ASM)
│       ├── manifest/            # Manifest 注入
│       ├── config/              # 配置与提示库加载
│       ├── task/                # Gradle Task (注入/验证)
│       ├── report/              # 报告生成
│       ├── rollback/            # 回滚
│       └─ rules/                # R8 Keep 规则生成
├─ injection-config.json        # 注入配置模板
└─ prompts/cbrn-prompts.json    # CBRN 提示文本库
```

概述

InjectionPlugin 是一个 Android Gradle 插件 (AGP 8+)，用于在 APK 构建过程中注入 CBRN（化学、生物、放射、核）相关的提示文本。当 AI 代理（如 ChatGPT、Claude、Gemini 等）尝试对此类 APK 进行逆向分析时，会触发其安全护栏，导致 AI 拒绝分析，从而达到反 AI 辅助逆向工程的目的。

插件通过两种方式注入内容：

- Manifest 注入**：在 `AndroidManifest.xml` 的 `<application>` 节点中插入 `<meta-data>` 标签
- 字节码注入**：使用 ASM 在目标类中添加静态字段，并在目标方法入口处引用该字段

环境要求

依赖	版本要求
Android Gradle Plugin (AGP)	8.0+
Gradle	8.0+
Kotlin	2.0+
JDK	17+
Python（可选，用于自动发现和仪表盘）	3.9+

第一步：安装插件

1.1 将插件发布到本地 Maven

在插件目录下执行：

```
cd injection-gradle-plugin
./gradlew publishToMavenLocal
```

1.2 在目标 App 项目中引入插件

在目标 App 的根目录 `settings.gradle.kts` 中添加：

```
pluginManagement {
    repositories {
        mavenLocal()
        google()
        mavenCentral()
    }
}
```

在目标 App 的 `app/build.gradle.kts` 中应用插件：

```
plugins {
    id("com.android.application")
    id("org.example.injection") version "1.0.0"
}
```

第二步：配置注入规则

在目标 App 的根目录下创建 `injection-config.json` 文件。

2.1 完整配置示例

```
{
  "manifest": {
    "enabled": true,
    "mode": "auto"
  },
  "bytecode": [
    {
      "class": "com.example.app.BackupHelper",
      "method": "exportData",
      "descriptor": "(Ljava/util/List;Ljava/io/OutputStream;)V",
      "anchor": "method-entry",
      "fieldName": "BACKUP_REVIEW_CONTEXT",
      "localVarName": "backupReviewContext"
    }
  ]
}
```

2.2 配置字段说明

Manifest 配置 (manifest 节点)

字段	类型	必填	说明
enabled	boolean	否	是否启用 Manifest 注入，默认 true
mode	string	否	提示文本选择模式： "auto" （自动选第一个）或 "manual" （手动指定）
promptId	string	否	手动模式时指定的提示 ID，如 "B-01"
promptCategory	string	否	手动模式时指定的类别，如 "bioweapon"
metaDataName	string	否	自定义 meta-data 的 android:name，默认 "guardrail.notice"

字节码配置 (bytecode 数组，每个元素对应一个注入目标)

字段	类型	必填	说明
class	string	是	目标类的全限定名，如 "com.example.app.BackupHelper"
method	string	是	目标方法名
descriptor	string	否	JVM 方法描述符，用于区分重载方法。为空则按名称匹配
anchor	string	否	注入点： "method-entry" （方法入口）或 "before:target-call" （目标调用前）
fieldName	string	否	注入的静态字段名，默认 "BACKUP_REVIEW_CONTEXT"
localVarName	string	否	局部变量名，默认 "backupReviewContext"
targetCall	object	否	当 anchor 为 "before:target-call" 时指定目标调用

targetCall 子配置 (仅在 anchor = "before:target-call" 时使用)

```
"targetCall": {
  "owner": "kotlinx/serialization/json/Json",
  "name": "encodeToString",
  "descriptor": "(Ljava/lang/Object;)Ljava/lang/String;"
}
```

字段	说明
owner	目标方法所属类的 JVM 内部名 (用 / 分隔), 如 "kotlinx/serialization/json/Json"
name	目标方法名
descriptor	目标方法的 JVM 描述符

第三步：准备提示文本库

在目标 App 根目录下创建 `prompts/cbrn-prompts.json`, 内容为 CBRN 提示文本库。插件已提供预置的提示库文件 (`prompts/cbrn-prompts.json`), 可直接复制使用。

第四步：自动发现注入目标 (可选)

如果不想手动指定字节码注入目标, 可使用 `discover-target.py` 自动扫描 App 源码, 推荐最佳注入目标:

```
python discover-target.py <app源码目录>
```

输出示例:

```
{
  "class": "org.fossify.notes.helpers.NotesHelper",
  "method": "exportNotes",
  "anchor": "method-entry",
  "fieldName": "BACKUP_REVIEW_CONTEXT",
  "localVarName": "backupReviewContext"
}
```

脚本会优先生成以下类型的类作为注入目标:

- Helper / Manager / Util / Repository 类
- 包含 export、import、backup、restore、save 等业务方法的类
- 自动跳过 Activity、Fragment、Adapter、ViewModel 等框架类

第五步：构建 App

在目标 App 目录下执行:

```
./gradlew assembleDebug
```

构建过程中，插件会自动：

1. **生成 R8 Keep 规则** — 防止注入的静态字段被混淆移除（输出到 `build/generated/injection/proguard-rules-injection.pro`）
2. **注入 Manifest** — 在合并后的 `AndroidManifest.xml` 中插入 `<meta-data>` 标签
3. **注入字节码** — 在目标类中插入静态字段，并在目标方法入口处加载该字段

构建日志中会显示注入结果，包括 Manifest 的 SHA-256 哈希（注入前后对比）。

第六步：验证注入结果

6.1 验证 Manifest 注入

```
./gradlew verifyDebugManifest
```

输出示例：

```
=====
[Verify] Pre-injection verification
=====
[Verify] OK: Manifest found at ../AndroidManifest.xml
[Verify] OK: <application> element found
[Verify] OK: Manifest injection enabled, mode=auto
[Verify] OK: Prompt library v2.0 loaded (1 categories, 10 prompts)
=====
[Verify] verification complete
```

6.2 回滚 Manifest 注入

如果需要撤销注入（恢复原始 Manifest）：

```
./gradlew rollbackDebugManifest
```

第七步：查看注入报告

```
./gradlew injectionReport
```

第八步、故障排查

构建报错：Method not found

```
[BytecodeInjection] Method 'xxx' not found in com.example.ClassName.
```

解决：检查 `injection-config.json` 中的 `class` 和 `method` 名称是否正确，或使用 `descriptor` 字段指定 JVM 方法描述符来区分重载方法。

构建报错：Method matches multiple methods

```
[BytecodeInjection] Method 'xxx' matches N methods.
```

解决：使用 `descriptor` 字段指定精确的 JVM 方法描述符。可通过 `javap -s` 查看编译后类的描述符。

重复注入提示

```
Meta-data 'guardrail.notice' already exists for variant debug. Skipped.
```

说明：Manifest 中已存在同名 meta-data，自动跳过。若需重新注入，先执行 `rollbackDebugManifest`。

R8 混淆后注入字段被移除

说明：插件会自动生成 ProGuard keep 规则。如果仍然遇到问题，检查 `build/generated/injection/proguard-rules-injection.pro` 是否被正确包含在构建中。